

Seal Native Engine vs llama.cpp — Vulkan Head-to-Head

Model: Qwen3.6-35B-A3B Q4_K_M | Hardware: Intel Arc B70 Pro 32GB (BMG G31) | Backend: Vulkan (both engines) | Date: 2026-05-01

Methodology: Both engines loaded the same GGUF file (`Qwen3.6-35B-A3B-Q4_K_M.gguf` , 19.70 GiB) on the same physical GPU (Vulkan0 = BMG G31, 28,748 MiB free before load). llama.cpp numbers from `llama-bench -p 512 -n 128 -r 3` . Seal numbers from `seal-native-infer --backend vulkan --max-tokens 128 --repeat 3 --warmup-tokens 4` with `SEAL_PROFILE_KERNELS=1` . Both runs were isolated — no concurrent processes. Seal prompt was 115 tokens (engineering question); llama.cpp used a synthetic 512-token fill.

1. Benchmark Results

17.1

Seal decode tok/s
tg128 steady-state

39.5

llama.cpp decode tok/s
tg128 ± 0.10

18.3

Seal prefill tok/s
pp115 steady-state

885

llama.cpp prefill tok/s
pp512 ± 5.26

Metric	Seal Native Engine (Vulkan)	llama.cpp (Vulkan)	Ratio
Decode throughput	17.1 tok/s (tg128, r3)	39.5 tok/s (tg128, 3-run avg)	0.43× (2.3× slower)
Prefill throughput	18.3 tok/s (pp115)	885 tok/s (pp512)	0.021× (48× slower)
Prefill token count (test)	115 tokens	512 tokens	—
GPU utilization (decode)	~3.9% (530ms GPU / 13,758ms wall for 243 tokens)	~60-80% (estimated, no profiling available)	~15× lower
Dispatches per token	612 dispatches / token (148,716 / 243 tok)	~10-50 estimated (graph batch per token)	~12-60× more
Syncs (fence waits) per token	1 sync/token — but 612 dispatches queued per sync	1 graph submit/token (all ops in one VkSubmit)	Same count; different barrier density
Model load time	7,609 ms (mmap → VRAM streaming)	~3,000-5,000 ms (estimated)	—
Warmup cost	6,394 ms (4 tokens primes pipelines)	None (JIT compiled per graph node)	—

Root cause of the 2.3× decode gap: Seal dispatches 612 Vulkan compute shaders per token, each preceded by pipeline barriers. GPU executes ~2.2ms of compute per token but wall time is ~58ms — only **3.9% GPU utilization**. The other 96% is barrier/submission overhead: each barrier round-trip is ~88µs, but each kernel runs only ~4µs. llama.cpp builds the entire token's computation graph into a single command buffer and submits it atomically — far fewer synchronization points.

Prefill gap (48×) is worse than decode: llama.cpp uses a tiled flash attention kernel for T>1 (prefill), reducing attention complexity. Seal uses naïve O(T × ctx) attention for prefill — every attention matrix multiply gets a separate dispatch with barriers. At 512 tokens this is ~262,000 serial attention ops. Fix: implement tiled prefill attention (flash-attn T>1 path).

2. Execution Model Comparison

Seal Native Engine — Imperative Dispatch

```

seal-native-infer → generate_with_backend()
| Loop: prefill tokens, then decode tokens
| Each token:
▼
LoadedModel::forward(&mut backend, token, pos)
| Called in model-registry/src/lib.rs
| Iterates 40 layers directly in Rust
▼
For each layer:
  backend.rmsnorm(...) → vkCmdDispatch ①
  backend.matmul_q4k(...) → vkCmdDispatch ②
  backend.matmul_q4k(...) → vkCmdDispatch ③
  backend.matmul_q4k(...) → vkCmdDispatch ④
  backend.matmul_q4k(...) → vkCmdDispatch ⑤
  backend.rope(...) → vkCmdDispatch ⑥
  backend.attention(...) → vkCmdDispatch ⑥
  ...
  [612 total dispatches per token]
▼
backend.sync() → vkQueueSubmit + fence wait
| Submits ALL pending commands, reads back logits
▼
sampler picks next token ID

```

Buffer system: deferred buffer map (`HashMap<*const f32, vkBuffer>`) keyed on host slice pointer. GPU tensors are `DEVICE_LOCAL`; `prep_input()` resolves by pointer lookup. Host-visible staging only for uploads and the final readback.

llama.cpp — Graph-Based DAG

```

llama_decode(ctx, batch)
|
▼
llm_build_qwen2moe()
| Builds ggml_tensor DAG — NO execution yet
| Every op is a node: ggml_mul_mat, ggml_add,
| ggml_rms_norm, ggml_rope, ggml_flash_attn_ext,
| ggml_cpy, ggml_silu, ggml_soft_max
▼
ggml_backend_sched_graph_compute()
| Assigns each node to a backend by tensor location
| Vulkan backend: GEMV, GEMM, attn, norm, rope
| CPU backend: fallback for unsupported ops (top-k)
| Entire graph submitted as one command stream
▼
Vulkan backend executes all nodes
| Single VkSubmit per token with all ops pipelined
| Barriers only where data dependencies require
▼
Logits in output buffer → sampler

```

Buffer system: `ggml_backend_buffer` allocates `DEVICE_LOCAL` tensors for weights. KV cache is a pre-allocated buffer (`struct llama_kv_cache`). Scheduler handles memory planning for all intermediate tensors automatically.

3. Model Architecture: Qwen3.6-35B-A3B Layer Structure

Seal's Qwen3.6-35B-A3B Forward Pass

Config (loaded from GGUF):

40 layers, 2048 hidden, 16 heads, 2 kv-heads
head_dim=256, vocab=248,320
rope_theta=10,000,000, rms_eps=1e-6
DeltaNet layers: 30 (linear-attn SSM)
Full-attn MoE layers: 10
MoE: 60 experts, top-2 active + 1 shared

Input embeddings [T, 2048]

▼ (for each of 40 layers)

SSM LAYER (30 layers):

① attn_norm rmsnorm

② SSM step:

conv1d + silu (1D causal conv)
q_gate_split (split Q/gate)
ssm_gdn_chain (state update)
sigmoid_mul_inplace (gate)
attention_split (output)

③ residual add + ffn_norm fused

④ MoE FFN:

moe_qx_topk_fused (router)
gate_up_swiglu (fused, 1 disp)
moe_accumulate_add_rmsnorm

FULL-ATTN MoE LAYER (10 layers):

① attn_norm rmsnorm

② GQA Attention:

Q,K,V projections (Q4_K GEMV)
rmsnorm_batched_heads (QK norm)
RoPE (NeoX style)
kv_append_pair (KV cache write)
flash-attn FA-2 split-K (T=1)
O projection (FP16 GEMV)

③ residual add + ffn_norm

④ MoE FFN: same as SSM layer

▼

Final norm (rmsnorm)
LM head: matmul_q6_k_gemv (248,320 vocab)
Logits [248,320] → sampler

llama.cpp's Qwen3-MoE Forward Pass

Input embeddings [T, hidden]

▼ (for each of N layers)

LAYER (standard transformer MoE):

① attn_norm ggml_rms_norm

② Self-Attention:

Q = ggml_mul_mat(x, W_q)
K = ggml_mul_mat(x, W_k)
V = ggml_mul_mat(x, W_v)
Q,K = ggml_rope(Q), ggml_rope(K)
[--flash-attn]:
ggml_flash_attn_ext(Q,Kc,Vc)
[without]:
scores = Q × K_cache^T
softmax(scores + causal_mask)
out = scores × V_cache
O proj = ggml_mul_mat(out, W_o)

③ Residual add ggml_add

④ ffn_norm ggml_rms_norm

⑤ MoE FFN:

Router → logits [T, E]
top-k + softmax + renorm
← CPU-side (tensor is small)
Per expert: gate/up/silu/down
separate ggml_tensor nodes
Shared expert: weight=1

⑥ Residual add ggml_add

▼

Final norm → lm_head → logits → sampler

Note: llama.cpp does not model DeltaNet SSM layers — it sees Qwen3-MoE as a standard transformer. DeltaNet is unique to the IE/Seal implementation.

4. KV Cache Implementation

Seal KV Cache

```
struct KvCache {
    // Per full-attn layer (10 layers):
    k: Vec<f32> // [max_seq, n_kv_heads, head_dim]
    v: Vec<f32> // [max_seq, n_kv_heads, head_dim]
    // Default: FP32 host storage
}
```

TurboQuant mode (--turboquant-kv):
 4-bit K + 4-bit V packed
 Randomized Hadamard rotation
 Groupsize=16, per-group scale (FP16)
 ~4x compression vs FP32

Write path (SPLIT – BUG):
 Decode T=1:
 kv_append_pair kernel writes new KV
 TurboQuant: quantizes inline
 Prefill T>1:
 ⚠ KV rows written as FP32 only
 TurboQuant NOT applied during prefill
 → KV cache is FP32 for prompt tokens,
 INT4 only for generated tokens

Read path:
 kv_append_pair reads back for attention
 attention_split reads cached pairs
 No flash attention for T>1 prefill
 Naïve $O(T \times \text{ctx})$ attention computation

Management:
 Sequential append only
 No defragmentation
 Cache invalidated on new context start

⚠ **Prefill KV quantization bug:** TurboQuant only activates on the decode path. Prefill tokens land in FP32 KV cache regardless of --turboquant-kv flag. Unified write path needed (same as llama.cpp's ggml_cpy approach).

llama.cpp KV Cache

```
struct llama_kv_cache {
    // Per layer:
    k: [n_kv_heads, max_ctx, head_dim]
    v: [n_kv_heads, max_ctx, head_dim]
    dtype: configurable (default FP16)
}
```

Default dtype: FP16 (GGML_TYPE_F16)
 Quantized options:
 -ctk q8_0 → Q8_0 keys (1 scale/32 elem)
 -ctv q8_0 → Q8_0 values
 -ctk q4_0 → Q4_0 keys (more compression)

Write path (UNIFIED – correct):
 T=1 and T>1 BOTH use ggml_cpy:
 ggml_cpy(K_new[T, kv_h, hd],
 k_cache[slot..slot+T, kv_h, hd])
 If dtype=FP16: straight copy
 If dtype=Q8_0: FP16→Q8_0 quantize inline
 1 scale per 32 elements during write
 Same kernel handles both prefill and decode
 → no prefill KV quantization bug

Read path:
 --flash-attn: flash_attn_ext tiles K/V
 inline dequantization during reads
 causal mask inside kernel
 handles T=1 AND T>1 shapes
 Without flash-attn: ggml_mul_mat dequant

Management:
 llama_kv_cache_find_slot() – find free slots
 llama_kv_cache_update() – defragment
 Cell-based: supports multi-turn reuse

✓ **Unified write path** means KV quantization is consistent between prefill and decode. No split-path bugs.

5. T=1 (Decode) vs T>1 (Prefill) Handling

Seal

Decode (T=1):
 GEMV path: each weight matrix uses dedicated GEMV kernel (M=1)
 Attention: FA-2 split-K custom kernel
 CHUNKS_PER_WG=8 super-chunk amort.
 SLM tiling, ~92% BW efficiency
 KV write: kv_append_pair (correct)
 Dispatch count: ~612 per token

Prefill (T>1):
 GEMM path: ??? – weight multiplies use same GEMV kernel even for T>1 (no M>1 GEMM kernel yet)
 Attention: NAïVE $O(T \times \text{ctx})$
 no tiling, no flash-attn equivalent
 separate dispatch per attention step
 → 48× slower than llama.cpp
 KV write: appends FP32 rows
 TurboQuant NOT applied ← bug

Key kernel: forward pass calls backend.matmul_qXk_gemv() regardless of batch size T. No T>1 dispatch path.
 This is why prefill is so slow.

llama.cpp

Decode (T=1):
 GEMV path: ggml_mul_mat with M=1
 dispatches mul_mat_vec_qX_K kernel
 inline dequantization
 Attention: flash_attn_ext (if --flash-attn)
 or naive SDPA (without flag)
 $Q[1, h, d] \times K_{\text{cache}}^T$ – trivial causal

Prefill (T>1):
 GEMM path: ggml_mul_mat with M>1
 DISPATCHES DIFFERENT KERNEL (tiled GEMM)
 mul_mat_q4_K – tiled, batched
 FP16×FP16 → DPAS/joint_matrix (coopmat)
 Attention: SAME flash_attn_ext kernel
 shape drives inner tiling
 $Q[T, h, d] \times K_{\text{cache}}^T$ – causal mask inside
 $O(T \times \text{ctx} / \text{tile}^2)$ with tiling
 → 885 tok/s on pp512

KV write: same ggml_cpy path for T=1/T>1
 Unified – no prefill quantization bug.

Shape dispatch happens transparently:
 ggml_sycl_mul_mat / ggml_vk_mul_mat
 selects kernel at RUNTIME based on M.

6. MoE Expert Routing

Seal — Fused GPU Router

Seal routes MoE entirely on GPU:

```
ws_x_norm [1, 2048]
↓
moe_qx_topk_fused kernel:
  Router matmul (Q4_K GEMV): x × W_router
    → logits [1, 60] (60 experts)
  Top-2 selection inline
  Softmax + renorm on GPU
  expert_ids [1, 2] INT32
  expert_weights [1, 2] FP32

For top-2 active experts + 1 shared:
  matmul_q4_k_moe_indirect_dp4a_gate_up_swiglu_sg32
    Fused: gather + gate matmul + up matmul
      + SwiGLU activation — 1 dispatch
  matmul_q4_k_moe_indirect_lcpp (down proj)
  moe_accumulate_add_rmsnorm
    Fused: weighted accumulate + add residual
      + RMS norm — 1 dispatch

Total MoE dispatches per token:
  moe_qx_topk_fused:           1 dispatch
  gate_up_swiglu (experts):     1 dispatch
  down proj (experts):         1 dispatch
  accumulate+add+norm:         1 dispatch
  = ~4-6 dispatches for full MoE FFN
```

Seal advantage: FUSED kernels cut the per-op overhead that llama.cpp has.

llama.cpp — Graph Nodes Per Expert

llama.cpp routes MoE via graph nodes:

```
ws_x_normed [T, hidden]
↓
ggml_mul_mat(x, W_router[E,H])
→ logits [T, E] FP32

Top-k + softmax + renorm:
  CPU-side for most backends
  (tensor is small: [1, 60])
  expert_ids [T,k] INT32
  expert_weights [T,k] FP32

For each active expert e:
  gather → x_e [n_tok, hidden]
  gate_out = ggml_mul_mat(x_e, W_gate_e)
  up_out   = ggml_mul_mat(x_e, W_up_e)
  h_out    = ggml_mul(ggml_silu(gate_out), up_out)
  down_out = ggml_mul_mat(h_out, W_down_e)
  out += weight_e * down_out
```

Each of the above = separate graph node
= separate Vulkan dispatch
No fused gate+up+silu kernel
No fused accumulate+norm

For top-2 from 60 experts:
~10-15 separate dispatches for MoE FFN

llama.cpp limitation: graph approach means no mandatory fusion — each op is its own kernel launch. Seal's fused MoE is better.

7. Kernel Inventory: Side by Side

7a. Seal Native Engine — Measured Kernel Profile

Steady-state run (repeat 3 of 3): 115 prefill + 128 decode = 243 total tokens. Total GPU time: 530.4 ms across 148,716 dispatches.

Kernel Name	Calls	Total ms	Avg μ s	% GPU time	What it does
matmul_q4_k_gemv_lcpp	24,300	92.1	3.79	17.4%	Q4_K GEMV for attention Q/K/V/O and FFN weights (dense layers)
moe_qx_topk_fused	9,720	46.5	4.78	8.8%	Router logits + top-k select + softmax renorm (fused, GPU-side)
matmul_q4_k_moe_indirect_dp4a_gate_up_swiglu_sg32	9,720	41.1	4.23	7.8%	MoE expert gate + up projection + SwiGLU (fused, indirect GEMV)
matmul_f16w_gemv	9,720	39.1	4.03	7.4%	FP16 weight GEMV (O projection, shared expert output)
moe_accumulate_add_rmsnorm	9,720	38.6	3.97	7.3%	Weighted expert accumulate + residual add + RMS norm (fused)
add_rmsnorm	9,720	38.3	3.94	7.2%	Residual add + RMS normalization (fused elementwise)
ssm_gdn_chain	7,290	36.5	5.00	6.9%	DeltaNet SSM state-space chain (recurrent state update, 30 layers)
matmul_q6_k_gemv_v2_gls1	9,720	36.3	3.73	6.8%	Q6_K GEMV v2 (faster path, attention output gate weights)
ssm_conv1d_silu	7,290	28.4	3.89	5.3%	SSM 1D causal convolution + SiLU (DeltaNet input projection, 30 layers)
matmul_q6_k_gemv_v1_sg32 ⚠	243	21.98	90.47	4.1%	LM HEAD ONLY (248,320 vocab) — wrong kernel! v1 at 90 μ s vs v2 at 3.73 μ s = 24 $\times$ regression. Should route to v2 or lcpp_full path.
matmul_q6_k_moe_indirect_lcpp	4,860	21.7	4.46	4.1%	MoE expert down projection Q6_K (indirect gather GEMV)
matmul_q4_k_moe_indirect_lcpp	4,860	19.9	4.09	3.7%	MoE expert down projection Q4_K (indirect gather GEMV)
attention_split	2,430	11.8	4.83	2.2%	FA-2 split-K attention (full-attn layers, T=1 decode)
rmsnorm_batched_heads	4,860	10.7	2.20	2.0%	Per-head RMS norm on Q/K before attention (QK norm)
kv_append_pair	2,430	9.53	3.92	1.8%	Write K/V pair to KV cache (decode, full-attn layers)
sigmoid_mul_inplace	2,430	9.48	3.90	1.7%	SSM gate: sigmoid(gate) $\times$ state (DeltaNet gating)
q_gate_split	2,430	9.20	3.78	1.7%	Split Q and gate channels from combined projection (SSM layers)
matmul_q4_k_ssm_3way	7,290	8.46	1.16	1.6%	SSM 3-way projection (A/B/C matrices for DeltaNet state)
matmul_q4_k_dp4a_gate_up_swiglu_sg32	9,720	5.96	0.61	1.1%	Shared expert gate+up+SwiGLU (fused, shared expert path)
dot_sigmoid	9,720	4.74	0.49	0.9%	Sigmoid activation on router weights (MoE routing gate)
rmsnorm_parallel	243	0.38	1.56	0.1%	Final layer norm before LM head

7b. llama.cpp Vulkan — Kernel Inventory (Structural)

llama.cpp Vulkan kernel names from `ggml-vulkan.cpp`. No per-kernel profiling was run in this session — avg_us and % are not available. Shape dispatch (M=1 $\rightarrow$ GEMV, M>1 $\rightarrow$ GEMM) is automatic at runtime.

Kernel Name	Status	Used For	Notes
mul_mat_vec_q4_K_f32	✅ Active	GEMV Q4_K M=1 decode hot path	Inline dequant; selected when M=1

<code>mul_mat_vec_q6_K_f32</code>	✔ Active	GEMV Q6_K M=1	LM head uses this for Q6_K weights
<code>mul_mat_q4_K</code>	✔ Active	GEMM Q4_K M>1 prefill	Tiled; triggered when M>1 (prefill batch)
<code>mul_mat_f16</code>	✔ Active	FP16×FP16 GEMM	coopmat path for FP16 prefill GEMM
<code>flash_attn_ext</code>	✔ Active (--flash-attn flag)	SDPA — handles T=1 and T>1	Unified; shape drives inner tile size
<code>soft_max</code>	✔ Active (without --flash-attn)	Naive SDPA softmax	Fallback when flash-attn not enabled
<code>rms_norm</code>	✔ Active	LayerNorm	Per-row workgroup reduction
<code>rope_neox</code>	✔ Active	RoPE encoding	NeoX style (Qwen uses this)
<code>silu_back / mul</code>	✔ Active	SiLU activation for SwiGLU	Separate node — not fused with gate
<code>add</code>	✔ Active	Elementwise add (residuals)	Separate dispatch per residual connection
<code>cpy_f32_f16</code>	✔ Active	KV cache write FP32 → FP16	Unified path for T=1 and T>1
<code>cpy_f16_q8_0</code>	✔ Active (-ctk q8_0)	KV cache write FP16 → Q8_0	1 scale per 32 elements; block-level
DeltaNet SSM kernels	✘ Not present	—	Non-standard arch; not in llama.cpp
Fused MoE gate+up+swiglu	✘ Not present	—	Each op is a separate graph node
Fused accumulate+add+norm	✘ Not present	—	Separate ggml_add + ggml_rms_norm nodes

8. Dispatch Overhead Analysis

Metric	Seal (measured)	llama.cpp (estimated)
Dispatches per decode token	~612 (148,716 dispatches / 243 tokens)	~20-60 (graph nodes per token)
Avg GPU work per dispatch	530.4ms / 148,716 = 3.6 µs/dispatch	~50-100 µs/dispatch (estimated)
Total GPU work per token (decode)	~2.18 ms GPU / 56.6 ms wall	~15-20 ms GPU / ~25 ms wall (est.)
GPU utilization	3.9% (530ms GPU / 13,758ms wall)	~60-80% (estimated)
Inter-dispatch barrier overhead	~88µs per barrier (56ms / 612 barriers)	~0.4ms per barrier (25ms / 60 dispatches)
Fence waits per token	1 (all 612 dispatches in one command buffer)	1 (entire graph in one submit)

The 3.6µs-per-kernel problem: Seal's kernels are individually fast (3-5µs each at production shapes), but every kernel is preceded by a full pipeline barrier (`VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT`). The GPU must drain the previous shader invocation before starting the next. On BMG G31 this round-trip costs ~80-90µs wall-clock even though the GPU only executes for ~4µs. The fix is not to remove barriers (correctness requires them between dependent ops) but to reduce the number of independent dispatch calls by fusing more ops into single kernels — which is exactly what MK-3 and MK-4 megakernels target.

9. Top Opportunities to Close the Gap

#	Fix	Expected Impact	Complexity
P0	LM head kernel routing — <code>matmul_q6_k_gemv_v1_sg32</code> fires at 90µs avg for the 248,320-vocab LM head. Should route to <code>v2_glsl</code> (3.73µs) or <code>lcpp_full</code> . Fix the shape-dispatch condition in <code>compute.rs</code> .	Removes 22ms/token of unnecessary GPU time (4.1% of GPU time on wrong path). Direct 5-8% decode speedup.	Low — routing logic change, no shader work
P1	MK-3/MK-4 megakernels — Fuse the SSM layer (~7 dispatches) and the MoE FFN (~4-6 dispatches) into single kernels. Target: reduce 612 dispatches/token to <100.	If barrier overhead drops proportionally: ~6× decode speedup → ~100 tok/s range.	High — new WGSL/GLSL kernels; cross-WG sync
P2	Tiled prefill attention (flash-attn T>1) — Current naive O(T×ctx) attention for prefill. llama.cpp's <code>flash_attn_ext</code> handles T>1 with the same kernel. Need a WGSL tiled attention kernel for M>1 prefill.	At pp512: Seal currently ~5 tok/s estimated; tiled attn → 100-400 tok/s range.	High — new attention kernel for T>1 shapes
P3	Fix INT8/TurboQuant KV prefill bug — Unified write path for T=1 and T>1. Prefill tokens should write quantized KV rows through the same <code>kv_append_pair</code> path as decode.	Correctness fix — prefill KV cache is currently FP32 even with --turboquant-kv flag.	Medium — Rust code path change in model-registry
P4	M>1 GEMM kernels for prefill — Weight projections during prefill use the same M=1 GEMV kernels regardless of batch size. For T=512, need tiled GEMM (like llama.cpp's <code>mul_mat_q4_K</code> for M>1 with coopmat acceleration).	Prefill throughput scales with T for weight-bound ops.	High — new tiled GEMM shaders

10. Full Architecture Comparison Table

Aspect	Seal Native Engine (Vulkan)	llama.cpp (Vulkan)	Winner
Execution model	Imperative: kernels called directly from Rust forward pass	Graph-based: build DAG of ggml_tensor nodes, then schedule	🟢 llama.cpp (better for heterogeneous dispatch)
Backend	Single Vulkan (BMG G31 primary target)	Multi-backend: CPU, CUDA, Metal, SYCL, Vulkan	🟢 llama.cpp (portability)
Decode throughput	17.1 tok/s (measured)	39.5 tok/s (measured)	🟢 llama.cpp (2.3×)
Prefill throughput	~18 tok/s pp115 (measured)	885 tok/s pp512 (measured)	🟢 llama.cpp (48×)
GPU utilization	~3.9%	~60-80% (est.)	🟢 llama.cpp
Attention T=1 (decode)	FA-2 split-K, custom SLM tiling, super-chunk amortization, ~92% BW	flash_attn_ext or naive SDPA (~60-80% est.)	🔴 Seal (better custom attention kernel)
Attention T>1 (prefill)	Naïve O(T × ctx) — no tiling, no flash-attn	Same flash_attn_ext kernel; shape drives tiling	🟢 llama.cpp
KV cache quantization write	SPLIT path: decode writes quant, prefill writes FP32 — BUG	Unified ggml_cpy for T=1 and T>1 — no split bug	🟢 llama.cpp
KV cache management	Sequential append only; no defrag; multi-turn requires reset	Cell-based: find_slot() + defrag; multi-turn reuse	🟢 llama.cpp
MoE fused decode kernel	✅ gate+up+SwiGLU fused, accumulate+add+norm fused — ~4 dispatches/MoE FFN	Separate graph node per op per expert — ~10-15 dispatches/MoE FFN	🔴 Seal (better fusion)
MoE router location	GPU-side: moe_qx_topk_fused kernel	CPU-side (tensor is tiny [1,60])	Tie (both work; CPU is faster for tiny tensor)
Runtime kernel shape dispatch	Manual per call site — M=1 hardcoded in most paths	Automatic: M drives GEMV vs GEMM selection at runtime	🟢 llama.cpp
DeltaNet SSM layers	✅ Fully implemented (30 layers, custom kernels)	❌ Not supported (non-standard arch)	🔴 Seal (unique capability)
DECODE GEMV Q4_K BW efficiency	~92% (custom lcpp kernel)	~60-80% est.	🔴 Seal
LM head kernel routing	⚠️ Routes to v1 at 90µs — BUG (v2 exists at 3.73µs)	Correct shape dispatch to vec kernel	🟢 llama.cpp
Memory planning	Manual: deferred buffer map; allocations tracked by host ptr	Automatic: ggml_backend_sched plans all tensor allocations from graph	🟢 llama.cpp
CPU fallback	❌ No automatic fallback — unsupported op = crash	✅ Automatic: unsupported nodes fall back to CPU backend	🟢 llama.cpp

Summary: llama.cpp wins on throughput (2.3× decode, 48× prefill), GPU utilization, and correctness (no split KV path bug). Seal wins on custom attention kernel quality, MoE fusion depth, and DeltaNet support. The performance gap is almost entirely structural: 612 dispatches/token with 3.6µs average GPU work per dispatch vs llama.cpp's fewer, larger batched dispatches. MK-3/MK-4 megakernels are the highest-leverage path to close this gap. After megakernels, the LM head routing fix (P0) is the single-file change with fastest turnaround.

Generated 2026-05-01. Seal kernel data is measured (SEAL_PROFILE_KERNELS=1, timestamp queries, VK_QUERY_TYPE_TIMESTAMP). llama.cpp kernel names from ggml-vulkan.cpp source structure; per-kernel timing not measured this session. Performance numbers are from actual benchmark runs on the same hardware, same model, same session.